

GRATICL: A UNIFIED FOUNDATION MODEL FOR TABULAR AND HOMOPHILIC GRAPH IN-CONTEXT LEARNING

Anonymous authors

Paper under double-blind review

ABSTRACT

Tabular Foundation Models (TFMs) solve classification and regression tasks through in-context learning (ICL), without any gradient updates at test time. Almost all of them realize ICL through dense attention over the samples of a dataset, so that every sample can exchange information with every other sample, at a cost that grows quadratically with the size of the context. We ask whether this dense layout is necessary, or whether a much sparser one suffices. To study this, we reframe tabular classification as node classification in a sparse graph: each sample becomes a node, and a sparsely connected, label-informed graph determines which samples may exchange information, so that ICL is carried out by message passing instead of dense attention. We instantiate this idea in GraTICL, a Graph Attention Network trained on an unmodified, fully open tabular prior, which isolates the effect of sparsification from changes to the data-generating process. We find that the number of attention operations can be reduced substantially while retaining most of the predictive performance, suggesting that dense sample-to-sample attention is largely redundant. Because the graph is an explicit input rather than a fixed architectural choice, the model is additionally promptable through its connectivity, which opens up test-time optimization over the graph structure, and the same recipe carries over to node classification on graph data.

1 INTRODUCTION

Tabular Foundation Models (TFMs) have pioneered a shift in tabular learning by replacing traditional per-dataset fitting with in-context learning (ICL). Pre-trained on millions of synthetic datasets, TFMs effectively learn the underlying learning algorithm itself and achieve remarkable performance on classification and regression tasks without any gradient updates at test time.

Almost all current TFMs realize ICL through dense attention over the samples of a dataset: every test sample attends to every training sample, and training samples attend to each other. This design places no restriction on which samples may exchange information, but it makes the cost of a forward pass grow quadratically with the number of samples in the context. As a consequence, the very regime in which tabular models are most useful, namely datasets with many rows, is also the regime in which TFMs are most expensive to apply.

This raises the question we study in this work: *is a dense sample-to-sample attention layout actually necessary for in-context learning on tabular data, or can it be replaced by a much sparser one?* To answer it, we reframe tabular classification as node classification in a sparse graph. Each sample of a dataset, whether labeled or unlabeled, becomes a node, and a sparsely connected, label-informed graph determines which samples are allowed to exchange information. ICL is then carried out by message passing along these edges rather than by dense attention, which reduces the number of attention operations from quadratic to linear in the number of samples. Since this formulation conditions the model on an explicit graph, it applies unchanged to data that already comes with a graph structure.

Our study yields three findings. First, the number of attention operations required for tabular ICL can be reduced substantially without sacrificing much predictive performance, indicating that dense sample-to-sample attention is largely redundant. Second, because the graph is an explicit input

rather than a fixed architectural choice, the resulting model becomes promptable through its graph structure, which opens up test-time optimization over the connectivity pattern. Third, the same recipe transfers to node classification on graph data, where the sampled graph is simply replaced by the observed one.

We make the following contributions:

1. **Sparse Reformulation:** We reframe tabular in-context learning as node classification in a sparse, label-informed graph, replacing dense sample-to-sample attention by message passing with linear cost in the number of samples.
2. **Architecture & Method:** We introduce GraTICL, a GAT-based foundation model trained on an otherwise unmodified tabular prior, so that the effect of sparsification can be isolated.
3. **Empirical Validation:** We evaluate the effect of sparsification on tabular benchmarks, study the model’s promptability through the graph structure at test time, and demonstrate that the approach transfers to node classification.

2 GRATICL: UNIFYING TABULAR AND NODE CLASSIFICATION

In the following, the problem setup and our methodology are presented.

2.1 PROBLEM STATEMENT

We aim to solve an in-context learning problem in the tabular/graph domain: Assume a labeled set of support instances $\mathcal{D}_S = \{(\mathbf{x}, y)_{i=0}^n\}$, an unlabeled query instance $\mathbf{x}_q$, and a graph $G \in \mathcal{G}(\{1, \dots, |\mathcal{D}_S| + 1\})$ without self-loops where $\mathcal{G}(\cdot)$ is the set of all graphs connecting samples from $\mathcal{D}_S \cup \{\mathbf{x}_q\}$. The goal is to predict the query instance’s label y_q in one forward pass. That is, we aim to learn $p_\theta(y_q|\mathcal{D}_S, G, \mathbf{x}_q)$ where θ are parameters of a neural network. We further assume $\mathbf{x} \in \mathcal{X}_S^{d_S}$ for all feature vectors, regardless of whether from the support or query set, where $\mathcal{X}_S$ is the feature space. It is assumed that $\mathcal{X}_S$ can contain real-valued and discrete variables and be of different dimensions for different tasks. The labels $y \in \{1, \dots, C\}$ are assumed to represent one of C classes.

2.2 ARCHITECTURE

To learn $p_\theta(y_q|\mathcal{D}_S, G, \mathbf{x}_q)$, we propose to leverage an adaptation of Graph Attention Transformers (GATs) performing message passing among the instances connected in G . From now on, let us denote $\mathcal{D}_s$ as an $n \times p$ matrix $\mathbf{X}_S$ where p is the number of features.

Encoding. First, each cell of $x_{ij} \in \mathbf{X}_S$ is encoded into a d -dimensional vector $\mathbf{x}_{ij} = e(x_{ij})$ where $e: \mathbb{R} \rightarrow \mathbb{R}^d$ is a shared linear layer. This yields a tensor $\mathbf{X} \in \mathbb{R}^{n \times p \times d}$. Column/feature identity is encoded additively, i.e., $\mathbf{x}_i = \mathbf{x}_{ij} + \text{col}(j)$ for each encoded cell $\mathbf{x}_{ij}$. Analogously, label information is encoded additively for all support samples, i.e., $\mathbf{x}_{ij} = \mathbf{x}_{ij} + h(y_i)$ where $h: \mathbb{R} \rightarrow \mathbb{R}^d$ is a linear transformation. Finally, a learnable d -dimensional CLS token is appended as an additional dimension for each sample.

ICL via Message Passing. After encoding each cell, we perform message passing along the edges defined in G . Note that G contains exactly one node for each support and query sample. Additionally, column-wise attention is employed to capture dependencies across columns. Consider a sample $\mathbf{x}_i \in \mathbf{X}$ and let $\mathbf{X}_{\mathcal{N}}$ be the set of neighbors of $\mathbf{x}_i$ in matrix form. Note that $\mathbf{x}_i$ is a $p \times d$ matrix. Our message passing operation is performed for each feature dimension j independently and is defined as follows:

$$\mathbf{x}_{ij} = \text{LayerNorm}(\mathbf{x}_{ij} + \text{MLP}(\text{CrossAttn}(\mathbf{x}_{ij}, \mathbf{X}_{\mathcal{N}}))) \quad (1)$$

To capture feature correlations, a standard multi-head attention along the feature dimension is employed:

$$\mathbf{X} = \mathbf{X} + \text{MultiHeadAttn}(\mathbf{X}) \quad (2)$$

Finally, a soft-knn readout layer is used to compute $p_\theta(y_q|\mathcal{D}_S, G, \mathbf{x}_q)$ using the distance between the encoding given by the CLS token of the query and support instances. For this, for a query sample $\mathbf{x}_q$, the cosine distance to each support sample $\mathbf{x}_i$ is computed. This yields a vector of distances $\mathbf{v} = (\cos(\mathbf{x}_q, \mathbf{x}_1), \dots, \cos(\mathbf{x}_q, \mathbf{x}_n))$. Applying softmax gives us an affinity vector $\mathbf{a} = \text{softmax}(\mathbf{v})$ which we use to compute a weighted sum of one-hot encoded labels.

$$p_\theta(y_q|\mathcal{D}_S, G, \mathbf{x}_q) = \sum_{i=1}^n \mathbf{a}_i \cdot \text{OneHot}(y_i) \quad (3)$$

2.3 GRAPH CONSTRUCTION & PRIOR

Since our model is conditioned on both a labeled dataset and a graph structure, we need to extend the prior commonly used to train TFMs by also sampling meaningful graphs that enable in-context learning.

Graph Prior. To enable ICL while maintaining linear attention complexity, we construct sparse, directed, class-conditioned graphs $G = (V, E)$ over the set of batch instances. The node set $V = V_S \cup V_Q$ comprises $|V_S| = n_S$ labeled support samples and $|V_Q| = n_Q$ unlabeled test samples, where directed edges $u \rightarrow v$ permit node v to attend to node u . Edges are generated through two complementary procedures governing internal training connectivity and target test aggregation.

Internal training topology ($V_S \rightarrow V_S$) relies on symmetric directed pairs to encourage both intra-class representation clustering and inter-class boundary awareness. For each graph, we sample a global pair budget $B \sim U(n_S \cdot k_{\min}, n_S \cdot k_{\max})$ and allocate a targeted fraction α to cross-class connections and $(1 - \alpha)$ to same-class connections. Edges are drawn uniformly via stochastic index sampling, with any residual budget from saturated class pools reallocated to the remaining pool. Restricting node degrees within this bounded range maintains linear $\mathcal{O}(n_S + n_Q)$ global message-passing complexity, while the stochastic sampling acts as a graph-level regularizer across training iterations.

Target test connections ($V_S \rightarrow V_Q$) treat test nodes strictly as information sinks to prevent data leakage and isolate target predictions. For every test node $v \in V_Q$ and each class label $c \in C$, we sample m training neighbors from class c without replacement, falling back to sampling with replacement only when class size is smaller than m . Enforcing zero outbound edge capacity for test instances ensures that test features never propagate into training representations or interfere across parallel test queries. Furthermore, guaranteeing a fixed context allocation of m neighbors per class ensures balanced representations during target aggregation, shielding the model from majority-class bias under severe training class imbalance.

Data Prior. We use the TabICLv2 data prior without modification. This choice provides a fully open-source tabular foundation-model reference: both the released TabICLv2 weights and the data prior used for its training are available. Holding the prior fixed allows us to systematically compare GraTICL with TabICLv2 and to meaningfully isolate the effect of sparsifying the computations through graph-based message passing.

2.4 TRAINING

GraTICL is pre-trained from scratch on synthetic datasets that are generated on the fly by the prior described above. We mostly reuse the TabICLv2 pre-training recipe and replace the dense ICL backbone by the graph-based message-passing backbone.

Objective. Each training example is a synthetic dataset that is split into a support and a query part. The model observes all features together with the support labels and predicts the query labels in a single forward pass; we minimize the cross-entropy between the predicted class distribution and the true query labels. Prior datasets contain between 1 and 256 features and at most 10 classes; the number of features seen by the model is padded to a common width within a micro-batch, and the per-dataset feature count is not passed to the model, so that all (padded) columns are treated uniformly.

Curriculum. Training proceeds in three stages with a batch size of 64 datasets throughout. Stage 1 runs for 500,000 steps on datasets of 2,048 samples with a support fraction drawn uniformly from $[0.3, 0.9]$, a peak learning rate of $8 \cdot 10^{-4}$, and gradient clipping at 10. Stage 2 runs for 40,000 steps on datasets whose size is drawn log-uniformly from $[400, 10,240]$ with a support fraction in $[0.79, 0.81]$, a peak learning rate of 10^{-4} , and gradient clipping at 10. Stage 3 runs for 10,000 steps on datasets of up to 60,000 samples (again log-uniform, starting at 400), a peak learning rate of $2 \cdot 10^{-5}$, and gradient clipping at 1. Each stage is initialized from the weights of the preceding stage; optimizer and scheduler state are not carried over across stages.

Optimization. We use the Muon optimizer with momentum 0.9, Nesterov updates, five Newton–Schulz iterations, and weight decay 0.01. The learning rate follows a cosine schedule with a warmup of 1% of the total steps, a single cycle, and a final learning rate of 10^{-7} . The batch of 64 datasets is split into micro-batches whose gradients are accumulated, so that the effective batch size is independent of the available memory. We use mixed-precision training throughout all training stages.

Model configuration. All experiments use an embedding dimension of 128. The column embedder and the row interactor each consist of three blocks with eight attention heads, and the ICL backbone consists of 12 message-passing blocks with eight heads. Note that the GAT-based ICL backend uses residual connections to prevent oversmoothing. Further, we sample 6 graphs per dataset from the graph prior, so that each sampled graph is shared by two consecutive message-passing blocks, and resampling across blocks exposes the model to several connectivity patterns per forward pass. This not only provides each training and test sample with different “views” w.r.t. the context, but also prevents oversmoothing. Feed-forward layers use an expansion factor of 2 with pre-normalization. Unless stated otherwise, the graph prior uses a degree budget of $k_{\min} = 8$ and $k_{\max} = 15$ pairs per support node, a cross-class fraction of $\alpha = 0.1$, and $m = 8$ support neighbors per class for each query node.

3 EMPIRICAL EVALUATION

4 RELATED WORK

5 CONCLUSION

REFERENCES

A APPENDIX

You may include other additional sections here.